

14. Volviendo a clases

BREVE RECAPITULACIÓN + GESTIÓN DE AMBIENTES + SALA DE CHAT

Programación I
Ingeniería Electrónica y Telecomunicaciones

Comisión 3
Profesor: Lautaro Linquimán
Universidad Nacional de Río Negro



Repaso express

¿Qué herramienta usamos?

1. Pedir datos hasta escribir “salir”.
2. Recorrer una lista de datos fijos.
3. Una función debe producir un resultado manipulable.
4. Una función debe recibir datos si los necesita.
5. Tenemos que guardar datos que no se repiten.
6. Tenemos que guardar datos que no se pueden modificar.

Analizando un poco de código

```
registros_temperatura = [  
    "FREY-12",  
    "OTTO;8°C",  
    "CATEDRAL;8",  
    "FREY;5"  
]  
  
total = 0  
  
for registro in registros_temperatura:  
    if registro.count(";") != 1:  
        print("Error en el registro ", registro, " se descarta")  
        continue  
  
    nombre, temperatura = registro.split(";")  
  
    if temperatura.isnumeric():  
        total += int(temperatura)  
  
print(total)
```

Analizar antes de ejecutar

- ¿Qué intenta calcular?
- ¿Qué línea puede fallar?
- ¿Qué registro lo rompe?
- ¿Por qué no conviene convertir directo?
- ¿Qué dato debería validarse antes?

Arrancamos sin ejecutar el código

Gestión de ambientes e instalación de paquetes

Ambientes virtuales

Un ambiente virtual es una carpeta del proyecto que usa una versión de Python y guarda allí los paquetes instalados para ese proyecto. Así evitamos modificar Python global o mezclar versiones entre proyectos.

1. Para crearlo: `python -m venv .venv`
2. Para activarlo:
 - Windows: `.venv\Scripts\activate.bat`
 - Linux/macOS: `source .venv/bin/activate`
3. Para desactivarlo: `deactivate`

Se crea una sola vez por proyecto. Si abrimos otra terminal, hay que activarlo de nuevo.

Instalación de paquetes

Dentro de nuestro ambiente virtual podemos instalar paquetes con: `pip install <paquete>`

Archivo requirements.txt

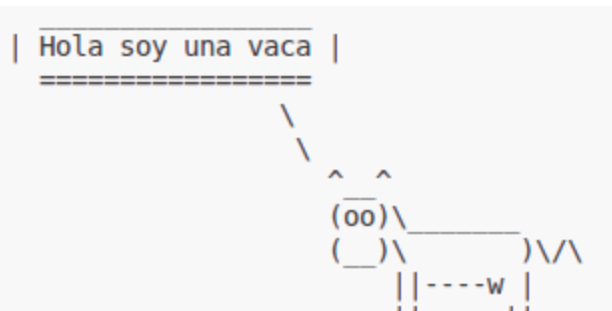
Los archivos requirements permiten definir listas de dependencias de paquetes. Cuando trabajemos en proyectos de software es el archivo nominal de dependencias. Para instalar el contenido de estos archivos usamos: `pip install -r requirements.txt`

Instalando nuestro primer paquete

1. Ejecutamos `pip install cowsay`
2. Ejecutamos `python`
3. Escribimos el siguiente código:

```
import cowsay  
  
cowsay.cow("Hola mundo")
```

Ejecutemos este código antes y después de instalar cowsay



Desafio

Construir una sala de chat

Requerimientos

- Se debe tener un programa donde poder enviar mensajes con el nombre de quien lo envió.
- Se debe tener un programa donde poder recibir mensajes con mi nombre.
- Se debe diferenciar entre los mensajes propios y los de otros.
- Los programas tienen que leer nuestro nombre de un archivo.

Creando ambiente

1. En un nuevo directorio vamos a crear un ambiente virtual
2. Copiar desde el repositorio `cliente.py` y `cliente_requirements.txt`
3. Ejecutar `pip install -r cliente_requirements.txt`

Creando nuestros programas

Modulo cliente.py

Esté modulo nos va a servir de base para implementar el siguiente programa.

Es modulo expone los metodos `enviar_mensaje` y `recibir_mensaje`, a partir de los cuales vamos a crear nuestros programas.

A codear

1. Crear un programa python que permita enviar mensajes hasta que escribamos salir.
2. Crear un programa python que permita recibir mensajes e imprimirlos en pantalla mostrando `usuario: texto`.
3. Ejecutar ambos programas en diferentes terminales.

Hay que modificar en cliente.py la variable `SERVIDOR` con datos que les voy a pasar.

Decorando nuestro programa

Vamos a pasar por tres maneras de decorar nuestro programa.

1. Utilizar cowsay para mostrar los mensajes.
2. Utilizar rich para mostrar los mensajes.
3. Utilizar streamlit para enviar los mensajes.

Armado de grupos para TP Integrador

https://docs.google.com/spreadsheets/d/1wL_SCAZfBA0m2F_d9-O5FGaUACXIVkYoXKM-IDBLsR4/edit?usp=sharing